

Home Task 2: Java Object-Oriented Programming Concepts

Question:

You have been tasked to develop a small Java program to simulate an online library system. The system should allow tracking and managing books and users. Your task is to implement the following requirements:

1. Create a **LibraryItem** Class:

- This class should have private attributes **title** and **author**.
- Implement a default constructor that sets the title to "Untitled" and the author to "Unknown."
- Provide a parameterized constructor to initialize these attributes.
- Implement getter and setter methods for both attributes.
- Implement a method **displayInfo()** that prints out the title and author.

2. Create a **Book** Class that Inherits from **LibraryItem**:

- Add a private attribute **ISBN** for storing the book's unique identifier.
- Implement a parameterized constructor that initializes all attributes (title, author, and ISBN) using the **super** keyword for the title and author.
- Provide a default constructor that initializes **ISBN** to "0000" and uses the default constructor of the parent class.
- Override the **displayInfo()** method to include the **ISBN** information.

3. Create a **EBook** Class that Inherits from **Book**:

- Add a private attribute **fileSize** to represent the size of the eBook in MB.
- Implement a parameterized constructor that initializes all attributes (title, author, ISBN, and fileSize).
- Provide a default constructor that sets **fileSize** to 0.0 and uses the default constructor of the **Book** class.
- Override the **displayInfo()** method to include the **fileSize** information.

4. Static Methods:

- In the **Book** class, implement a static method **bookCount()** that returns the total number of **Book** objects created. You need to keep track of the count using a static variable.

5. Method Overloading:

- In the **EBook** class, overload a method **download()** that can either:
 1. Download a file given its file name.
 2. Download a file given its file name and location.

6. Encapsulation:

- Ensure that all the attributes of your classes are private.
- Use getter and setter methods to access and modify the attributes, ensuring data encapsulation.

7. Multi-level Inheritance:

- Explain in your code comments how the classes `LibraryItem`, `Book`, and `EBook` demonstrate multi-level inheritance.

8. Program Execution:

- In your `main` method, create instances of `LibraryItem`, `Book`, and `EBook` using both default and parameterized constructors.
- Demonstrate the usage of getter and setter methods.
- Show how method overloading and method overriding work by calling the `download()` method for the `EBook` and `displayInfo()` method for each class.
- Track the number of books using the `bookCount()` static method.

// Class Demonstration: Inheritance Example (Employee System)

```
class Employee {  
    // Encapsulation  
    private String name;  
    private double salary;  
  
    // Default constructor  
    public Employee() {  
        name = "Unknown";  
        salary = 0.0;  
    }  
  
    // Parameterized constructor  
    public Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
  
    // Getters & setters  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public double getSalary() {  
        return salary;  
    }  
  
    public void setSalary(double salary) {  
        this.salary = salary;  
    }  
  
    public void displayInfo() {  
        System.out.println("Name: " + name);  
        System.out.println("Salary: " + salary);  
    }  
}
```

// Single Inheritance: Manager extends Employee

```
class Manager extends Employee {  
    private String managerId;  
    private static int count = 0;  
  
    // Default constructor  
    public Manager() {  
        super();  
        managerId = "M000";  
        count++;  
    }  
}
```

```

// Parameterized constructor
public Manager(String name, double salary, String managerId) {
    super(name, salary);
    this.managerId = managerId;
    count++;
}

// Static method
public static int managerCount() {
    return count;
}

// Method overriding
@Override
public void displayInfo() {
    super.displayInfo();
    System.out.println("Manager ID: " + managerId);
}

// Method overloading
public void assignTask(String task) {
    System.out.println("Assigned task: " + task);
}

public void assignTask(String task, String deadline) {
    System.out.println("Assigned task: " + task + " (Deadline: " + deadline + ")");
}

}

public class ClassDemo {
    public static void main(String[] args) {

        Manager m1 = new Manager();
        Manager m2 = new Manager("Alice", 75000, "M101");

        System.out.println("\n--- Manager 1 ---");
        m1.displayInfo();

        System.out.println("\n--- Manager 2 ---");
        m2.displayInfo();

        System.out.println("\n--- Task Assignment ---");
        m2.assignTask("Prepare Report");
        m2.assignTask("Prepare Report", "Friday");

        System.out.println("\nTotal Managers: " + Manager.managerCount());
    }
}

```